

The Core Web Vitals guide for business websites

What LCP, INP and CLS measure, what breaks them, and how to hold a supplier to them

Calwebtech

Custom websites, web applications and ecommerce for B2B companies
calwebtech.com

Last updated 15 September 2026

This guide is written for people who buy websites rather than build them, and for the developers who have to explain a failing report to them. It covers what each Core Web Vital measures, the thresholds published on web.dev, the handful of causes behind almost every failure on a business website, the order to diagnose them in, and the wording that turns "fast" into something you can check.

1. Three measurements, and what they are for

Core Web Vitals exist because the older measures of speed did not describe what visitors experience. Time to first byte tells you about the server, not about whether anyone could read the page. The vitals are deliberately user-centred: when did it appear, does it respond, and does it hold still.

Largest Contentful Paint measures when the largest element in the viewport has rendered. On a business site that is almost always the hero image, a video poster or a large headline. web.dev publishes 2.5 seconds or less as good, with 4 seconds as the boundary of poor.

Interaction to Next Paint measures the delay between an interaction and the next visible frame, taking the worst interactions across the visit rather than only the first. web.dev publishes 200 milliseconds or less as good, with 500 milliseconds as the boundary of poor.

Cumulative Layout Shift measures unexpected movement of visible content as a unitless score. web.dev publishes 0.1 or less as good, with 0.25 as the boundary of poor. All three are assessed at the seventy-fifth percentile of real visits, so a good average cannot hide a bad experience for a quarter of your visitors.

2. Field data and laboratory data

Two kinds of measurement circulate in these conversations, and confusing them wastes a great deal of time. Laboratory data comes from a tool running a single visit under simulated conditions: Lighthouse, PageSpeed Insights in its laboratory section, or a continuous integration check. Field data comes from real visits, gathered from real browsers.

Field data decides whether you have a problem. It reflects the phones your buyers actually own, the networks they are on and the pages they actually visit. Laboratory data is for diagnosis: it is repeatable, so you can change one thing and see the effect.

The most common surprise is a site that scores well in the laboratory and fails in the field. The usual explanations are that the test device is far faster than the average visitor's phone, that the test ran on the homepage while the traffic arrives on heavier templates, or that tags added through a tag manager are present in production and absent in the test.

Collect your own field data as well as reading the platform reports, because the platform report is aggregated and delayed. A small script that records the three vitals and sends them to your own analytics gives you the same numbers per template, immediately.

3. Diagnosing Largest Contentful Paint

Work in this order, because it is roughly the order of impact per hour spent. First, identify the element: browser tools will name it, and it is usually not what the team assumed. Second, check how it is delivered. A hero image should be sized for the viewport, served in a modern format, given explicit dimensions, and marked as high priority so the browser fetches it immediately rather than discovering it late.

Never lazy-load the largest element. Lazy loading is correct for images below the fold and actively harmful for the one at the top, because it delays the very thing being measured.

Third, look at fonts. Text that waits for a web font is text nobody can read yet. Preload the one or two faces the first screen needs, and allow the text to render immediately with a fallback rather than sitting invisible.

Fourth, look at the server response. If the HTML takes a second to arrive, everything after it starts late. Caching the rendered page for a short period, rendering on the server rather than in the browser, and serving static assets from an edge near the visitor all address this directly.

Fifth, background video. It should always have a poster image, should never be the largest contentful element, and should load after the page is usable. A video that blocks the hero is the most expensive decoration on the internet.

On images specifically, three details do most of the work. Serve several widths and let the browser choose, so a phone downloads a phone-sized file rather than a desktop one. State the sizes the image will occupy at each breakpoint, because without that the browser guesses and usually guesses high. And set explicit dimensions or an aspect ratio, which fixes layout shift at the same time.

Quality settings are worth a conversation with whoever signs off the design. The difference between a photograph at maximum quality and one at a sensible setting is often two thirds of the file size and invisible on a phone. Agreeing that once, as a rule in the media library, is better than arguing about it per image.

- Identify the actual LCP element before changing anything
- Size, format and prioritise it; remove any lazy-loading from it
- Preload critical fonts; let text render with a fallback
- Cache the HTML and serve assets from an edge
- Give video a poster and load it after the page is usable

4. Diagnosing Interaction to Next Paint

Interaction to Next Paint is a JavaScript problem in nearly every case. The browser runs your scripts on the same thread it uses to respond to taps, so a long task blocks the response. The metric takes the worst interactions of the visit, which means a page that is sluggish once is scored on that moment.

Start with an inventory of third-party scripts, because on most business sites they outweigh the site's own code. Chat widgets, heat-mapping tools, several analytics tags, advertising pixels, cookie banners and A/B testing scripts accumulate over years and nobody removes them. Every tag should have a named owner and a stated purpose, and anything without both should be removed for a fortnight to see whether anyone notices.

Then look at your own code. The question is not how fast the framework is but how much of the page needs to run in the browser at all. Content that only displays information can be rendered on the server and shipped as HTML. Genuinely interactive pieces, a calculator or a booking picker, can be loaded when they are needed instead of on every page.

Finally, look for long tasks: work that occupies the main thread for more than fifty milliseconds. Breaking large pieces of work into smaller ones, deferring what is not needed for the first interaction, and avoiding expensive layout recalculation in response to every scroll event are the standard remedies.

5. Diagnosing Cumulative Layout Shift

Layout shift is the easiest of the three to fix and the most embarrassing to leave. Every cause is the same: something appeared, and no space had been reserved for it.

Images and embeds need explicit dimensions or an aspect ratio, so the browser reserves the space before the file arrives. Advertisement and recommendation slots need a reserved minimum height. Banners, consent notices and promotional strips should be placed in space that already exists rather than inserted above content that then jumps down.

Fonts cause a subtler version: text rendered in a fallback and then re-rendered in a web font with different metrics moves everything below it. Matching the fallback metrics closely, or preloading the face, removes most of it.

Late-arriving components are the last category. A price, a stock level or a personalised block that loads after the page should occupy its final size from the start, showing a placeholder of the same dimensions rather than growing when the data arrives.

One rule prevents most new shifts: animate only opacity and transform. Animating height, top or margin moves everything around it, costs more to render, and is what makes a page feel cheap on a mid-range phone.

6. The cost of a page builder

A large share of failing business sites share one cause: a page builder or a theme bought for its demonstration content. The output carries markup for layout options nobody used, stylesheets loaded on every page, and a dozen scripts that arrive whether or not the page needs them.

This is not an argument that every site needs bespoke development. It is an argument for knowing what you are paying. A builder trades a slower, heavier page and a harder migration for faster initial assembly. If the site is small and rarely changes, that can be a fair trade. If the site is the main source of enquiries, it rarely is.

The alternative that keeps editors independent is content types: a service, a location, a case study, each with its own fields and its own template. Publishing stays a five-minute job, and the page weight is decided once by the template rather than by whoever built the page.

7. Make the thresholds a gate, not a goal

Performance work that is done once decays. Tags are added, images are uploaded at the wrong size, a new template arrives without the same care. The only durable approach is to check on every release and to fail the build when a threshold is breached.

On our own platform two gates run in continuous integration. A Lighthouse gate checks the vitals and the total page weight, and a separate budget checks our own client-side JavaScript per route, so that growth in the framework and growth in the page are never confused with one another. Both run on every pull request.

Field data sits alongside them as the real measure. Laboratory checks catch regressions early; the seventy-fifth percentile of real visits tells you whether your buyers are having a good time.

- Automated checks on every pull request, not once at launch
- Separate budgets for framework code and page code
- Field monitoring per template, at the seventy-fifth percentile
- A named owner for every third-party tag

8. What to write into a contract

Vague performance language is unenforceable. These are the clauses worth insisting on, in plain terms. The named templates — homepage, service page, listing page, case study, contact — will meet Largest Contentful Paint under 2.5 seconds, Interaction to Next Paint under 200 milliseconds and Cumulative Layout Shift under 0.1, measured at the seventy-fifth percentile of field data over twenty-eight days.

Add the method: checks run automatically on every release, results are reported to the client, and a regression is fixed within an agreed window. Add the exception honestly: third-party scripts the client adds after launch are outside the supplier's control, and the supplier will report their measured cost rather than quietly absorb it.

Finally, ask for the budget in kilobytes for client-side JavaScript on a marketing page, and a note of what the current figure is. A supplier who has never measured it will not be able to answer, which is itself the answer.

9. A worked order of operations

If you have a failing report and a fortnight, this is the order we would work in. Day one: collect field data per template and identify which templates fail and on which metric. Day two: identify the LCP element on each failing template and check its delivery.

Days three to five: image handling, font loading and priority hints. This is usually most of the Largest Contentful Paint problem and it rarely requires design changes.

Days six to seven: reserve space for everything that arrives late, and remove any animation of layout properties. This usually resolves Cumulative Layout Shift entirely.

Days eight to ten: the third-party script audit, with owners named and unclaimed tags removed. Measure again; this is often enough to move Interaction to Next Paint into the acceptable band.

Days eleven onwards: the architectural work, if it is still needed. Rendering on the server, cutting the amount of the page that runs in the browser, and loading heavy interactive pieces only where they are used. This is the expensive part, which is why it comes last.

10. Measuring the field data yourself

Platform reports are aggregated and delayed, which makes them a poor tool for judging a change you shipped last week. Collecting your own field data removes both problems, and it is a small piece of work.

Browsers expose the three vitals to a short script on the page. That script records the value, the template it came from and a coarse device category, and sends it to your own analytics. Nothing about an individual visitor needs to be stored, and no third-party service needs to be involved, which keeps it inside a consent policy that blocks analytics until permission is given.

Report it per template, at the seventy-fifth percentile, over a rolling four-week window. That matches the way the platform reports work, so the two do not disagree for uninteresting reasons, and it gives you a number per page type rather than one number for the site.

Watch the shape as well as the value. A metric that is good on average and terrible on a particular template is a specific, fixable problem. A metric that degrades slowly over months is usually accumulation: another tag, another embedded widget, another image uploaded at full resolution.

The one thing worth alerting on is a step change after a release. Everything else can be a weekly review.

11. Four beliefs that waste money

The first is that a perfect Lighthouse score is the goal. It is a diagnostic tool on a simulated device. Chasing the last few points on a laptop while the field data fails is optimising for the wrong audience.

The second is that a faster server fixes everything. Server time matters, but on most business sites the largest element is an image and the interaction delay is JavaScript. Moving to a bigger machine changes neither.

The third is that performance is a phase near the end of a project. Rendering strategy, image handling and how much runs in the browser are architectural decisions taken in week two. Retrofitting them after launch costs several times what building them in would have cost.

The fourth is that a plugin will fix it. Caching plugins help with server response and do nothing about a hero image four times larger than it needs to be, a font that blocks text, or a chat widget loading on every page. The work is specific, and so are the fixes.

- A laboratory score is a diagnosis, not a target
- Most LCP problems are images; most INP problems are scripts
- Rendering strategy is an early decision, not a late fix
- No plugin removes a script somebody chose to add

Checklist: questions for whoever builds your site

Ask these before signing. The answers, or the absence of them, tell you what you need to know.

- What are our field figures today, per template, at the 75th percentile?
- Which element is the LCP element on each key template?
- How much client-side JavaScript does a marketing page ship, in kilobytes?
- Which third-party scripts are loaded, who owns each one, and what does each cost?
- How are the thresholds checked on every release?

- What happens, contractually, when a release breaches them?

Sources

- Core Web Vitals on web.dev — <https://web.dev/articles/vitals>
- Largest Contentful Paint — <https://web.dev/articles/lcp>
- Interaction to Next Paint — <https://web.dev/articles/inp>
- Cumulative Layout Shift — <https://web.dev/articles/cls>

Who wrote this

Calwebtech builds custom websites, web applications and ecommerce for companies selling to other businesses. We publish our price bands, our process and our results, and we write these guides from the projects rather than from a content calendar.

If this raised a question about your own site, ask us: calwebtech.com/contact/

The whole argument of this guide is also published, free and without a form, at calwebtech.com/guides/core-web-vitals-guide/